Understand the Conveyor Program

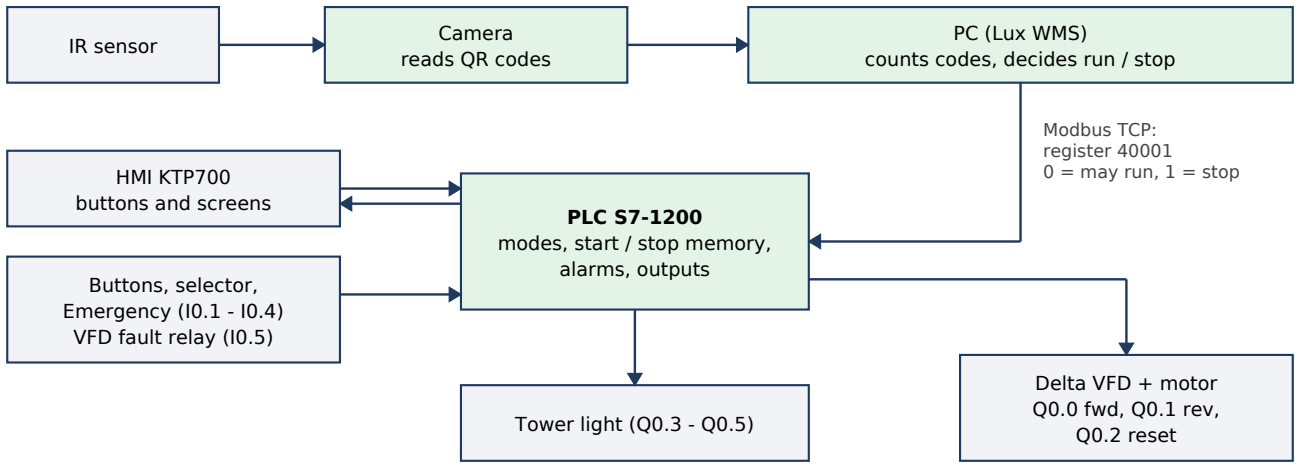
What every part means, why it is there, and how to design it yourself from the problem statement.
Companion to "Write the Conveyor Program Yourself" (Rev 3). Revision 2, 12 October 2026: checked against the old program exported by TIA Portal itself.

How to use this guide. The other guide tells you **what to click**. This one tells you **what it means and why**. Read parts 1 to 5 first (about one hour). Then follow the how-to guide, and each time it says "create ...", read the matching part here.
Everything about the old program (names, addresses, networks, order) was read from the old project file "latest plc". When I explain **why** the author did something, that is the usual reason in PLC work; the author did not write their reasons down. Where a choice has no technical reason, I say so.

When the how-to guide is at	Read here
Part 1 (project, hardware, system memory byte)	3.1, 3.3
Part 2 (PLC tags)	3.3, 4, 6
Part 3 (data blocks)	3.3, 7
Parts 4 to 8b (program blocks and ladder)	3.4, 3.5, 8, 9, 10, 11
Part 9 (OB1, compile)	2, 8
Part 10 (HMI)	12
Part 11 (download and test)	5 step 10, 10

1. The job of the PLC in this machine

Before any program, be clear about **who decides what**. In this machine the PLC is not the brain of everything. The camera reads the codes, the PC checks them, and the PLC only drives the belt and the lamps safely.

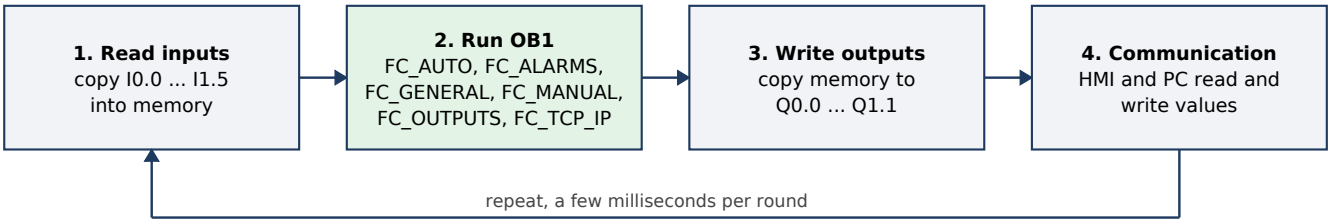


The PLC decides	The PLC does NOT decide
Which mode is active (AUTO / MANUAL)	Whether the QR codes are right (the PC does this)
Whether the auto cycle is started or stopped (START / STOP buttons)	When to trigger the camera (the sensor is wired to the camera)
Whether the belt may move (no Emergency, no fault)	Speed and ramps of the motor (set inside the VFD parameters)
Which alarms are active, and when they are cleared	

The PLC decides	The PLC does NOT decide
Which lamps are on	

2. How a PLC runs a program: the scan cycle

A PLC does not wait for a button like a PC program waits for a mouse click. It repeats the same four steps again and again, a few hundred times per second. One round is called a **scan** or **cycle**.



Three things follow from this, and they explain many details of the program:

1. **The program reads a photo of the inputs.** All blocks in one scan see the same input values, even if a button changes in the middle.
2. **A coil writes its result in every scan.** A normal coil is not "switched on once": it is recalculated every few milliseconds. If two networks write the same coil, the **last one wins**. That is why each output is written in one place only (part 9, FC_OUTPUTS).
3. **Order matters.** A block that runs early sees values that later blocks calculated in the **previous** scan. With a few milliseconds per scan this delay is too short to notice, but it is why the order in OB1 is worth thinking about (part 8).

3. Words you must know

3.1 Hardware and project words

Word	Meaning	In this project
Project	The TIA Portal file that holds everything: hardware, PLC program, HMI screens.	"latest plc" (old), "Conveyor_New" (yours)
Device	One piece of hardware in the project.	PLC_1 (the CPU) and HMI_1 (the panel)
CPU	The PLC itself: processor, memory, built-in inputs and outputs.	CPU 1214C DC/DC/DC: 24 V DC supply, 24 V DC inputs, 24 V DC transistor outputs
Firmware version	The software version inside the CPU or panel. The project must be set to a version the device can run.	PLC V4.4. HMI 17.0 (this is why the panel needed the image update)
IP address	The network address of a device. All devices must be in the same range (192.168.0.x) to talk.	.40 PLC, .41 HMI, .77 PC, .78 camera
Subnet PN/IE_1	The network line drawn in TIA between devices that talk to each other.	PLC and HMI are both on PN/IE_1

3.2 Addresses: how the PLC numbers its signals

Every signal has an address made of a **letter**, a **byte number** and a **bit number**. A byte is 8 bits, numbered 0 to 7.

Address	Read it as	Example in this project
%I0.3	Input, byte 0, bit 3	STOP button

Address	Read it as	Example in this project
%Q0.4	Output (Q = from German "Ausgang"), byte 0, bit 4	Red lamp
%M2.4	Memory bit (flag), byte 2, bit 4. Has no wire; it lives inside the PLC.	FAULT BIT
%IB0	Input Byte 0 = all 8 inputs I0.0 ... I0.7 together	The HMI input-status screen reads this
%DB2.DBX0.1	Data block 2, byte 0, bit 1 (DBX = bit)	VFD FAULT alarm
%DB2.DBW0	Data block 2, Word (16 bits) starting at byte 0 = bytes 0 and 1	All 16 alarm bits at once
%DB5.DBW0	Data block 5, word at byte 0	DATA1 = Modbus register 40001

The **%** sign means "this is an absolute address". Without it, TIA thinks you typed a name.

3.3 Data types, tags and memory

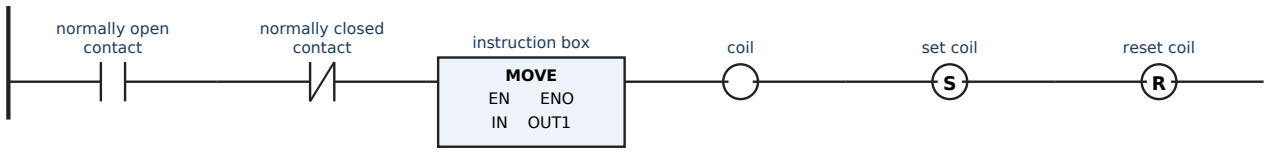
Word	Meaning	In this project
Bool	1 bit: 0 or 1 (FALSE / TRUE, OFF / ON).	All buttons, lamps, flags, alarms
Byte	8 bits.	IB0, QB0 for the HMI status lamps
Word	16 bits, used as a group of bits, not as a number.	ALARM WORD = 16 alarm bits
Int	Whole number, 16 bits, -32768 ... 32767.	AUTO STATUS (10 / 20 / 30), DATA1 ... DATA19
DInt / UDInt	Whole number, 32 bits. UDInt = no minus sign, up to about 4 billion.	STOP TIME (DInt), message counters (UDInt)
IEC_TIMER	The memory a timer needs (start time, preset, elapsed time, output).	STOP DELAY TIMER, SENSOR CHECK DELAY (old sensor cycle)
TCON_IP_v4	A ready-made group of values that describes one network connection.	MY CONNECT DATA, used by the Modbus server
Tag	A **name** for an address, so the program says "DI_CYCLE START PB" instead of "%I0.2". If the wire moves to another input, you change the address once and the whole program follows.	Three tag tables: DI (14), DO (10), Default (7 flags + system and clock tags)
PLC tag / HMI tag	PLC tags live in the PLC. HMI tags live in the panel and are linked to a PLC tag or address. The panel can only show or change PLC values through its HMI tags.	15 HMI tags in 3 tables: Default, IO, ALARMS (how-to 10.2)
Flag (M bit)	A Bool in the PLC memory with no wire. Used to pass a decision from one block to another.	M2.0 ... M2.6
System memory byte	A byte that the CPU fills by itself with useful bits: FirstScan (1 only in the first scan), AlwaysTRUE (always 1), AlwaysFALSE (always 0).	System byte at MB1: AlwaysTRUE = M1.2, AlwaysFALSE = M1.3. Clock byte also ON at MB0 (Clock_10Hz ... Clock_0.5Hz)
Data block (DB)	A named table of values in the PLC memory. Like a page in a notebook where a block keeps its data.	DB1, DB2, DB3, DB5, DB12 (yours) + DB4 (automatic)
Global DB	A DB that every block may read and write.	All five that you create
Instance DB	A DB that belongs to one call of a function block (FB) and holds its private memory. TIA creates it.	MB_SERVER_DB (DB4)
Offset	Where a value sits inside a DB, in byte.bit. 3.0 = byte 3, bit 0.	DATA1 at 0.0, DATA2 at 2.0 ...
Optimized block access	If ticked, TIA arranges the DB for speed and the values get **no fixed offset**. Then you cannot use addresses like DB2.DBW0 and the Modbus server cannot point into the DB.	Unticked on DB_HMI, DB_ALARMS, DB_AUTO, DB_CAMERA; DB_TCP_IP stays optimized
Start value	The value a DB entry has after the program is downloaded (and, if not retained, after power-on).	All 0. So DATA1 = 0 (run allowed) after power-on

3.4 Program blocks

Block	Meaning	In this project
OB (organization block)	A block the CPU calls by itself. **OB1 "Main"** is called in every scan. Everything you want to run must be called from OB1.	Main [OB1]: calls the six FCs
FC (function)	A block of program with **no memory of its own**. It reads and writes tags and global DBs. Called from OB1.	FC1 ... FC6
FB (function block)	A block of program **with its own memory** (instance DB). Used when the block must remember things between scans.	MB_SERVER is an FB from Siemens
Network	One drawing in a block, between the left rail and the coils. Good practice: one network answers one question.	FC_OUTPUTS has 6 networks: one per output
LAD / SCL	LAD = ladder diagram (pictures of contacts and coils). SCL = text, like a programming language.	Ladder everywhere; one SCL line in the old sensor cycle

3.5 Ladder instructions used

Imagine current flowing from the **left rail** to the right. A contact either lets it pass or blocks it. What reaches the right end decides the result.



Instruction	What it does	Used for in this project
Normally open contact -| |-	Passes when the tag is **1**.	"Is the selector at AUTO?"
Normally closed contact -|/|-	Passes when the tag is **0**.	"Is there NO fault?" (contact on FAULT BIT)
Contacts in a row	AND: all must pass.	Control ON **and** AUTO **and** no fault
Contacts one below the other (branch)	OR: one of them is enough.	START **or** already running (the latch)
Coil -( )-	Writes 1 when current arrives, 0 when it does not. Every scan.	Lamps, outputs, flags
Set coil -(S)-	Writes 1 when current arrives. Does nothing when it does not: the 1 stays.	Alarms that must stay until reset
Reset coil -(R)-	Writes 0 when current arrives.	Old sensor cycle
MOVE	Copies the value at IN to OUT1.	Write 0 to all alarms; write 10 / 20 to AUTO STATUS; connection settings
CMP ==, CMP <>	Compare two values: equal / not equal. Passes current when true.	"Is DATA1 = 0?", "Is any alarm bit set?"
ADD	Adds two numbers.	Message counters
TON	On-delay timer: output goes 1 after the input has been 1 for the preset time.	Old sensor cycle only
P_TRIG	Gives 1 for only one scan when a signal changes from 0 to 1 (rising edge).	Old sensor cycle only

4. The most common confusion: NC buttons and NC contacts

"Normally closed" appears in two places, and they are **different things**:

- The **button** on the panel door has a real contact block: NO (closes when pressed) or NC (opens when pressed). This decides the **wire**.
- The **contact symbol** in the ladder program only asks a question about the input bit: NO symbol = "is it 1?", NC symbol = "is it 0?".

Device	Contact block	Not pressed: input	Pressed: input	Program symbol used	Symbol passes when
START (green)	NO	0	1	NO -| |-	button pressed
STOP	NC	1	0	NO -| |-	button NOT pressed
Emergency	NC	1	0	NO -| |- in FC_GENERAL	Emergency NOT pressed (healthy)
Emergency	NC	1	0	NC -|/|- in FC_ALARMS	Emergency pressed (raise alarm)

Why are STOP and Emergency wired NC? Because a broken wire or a loose screw then looks like "STOP pressed" and the machine stops. With an NO button, a broken wire would make STOP impossible and nobody would know. This is called **fail-safe** wiring. That is why on a healthy machine the input lamps I0.3 and I0.4 are ON while nobody touches anything.

Safety note. In this design the Emergency button only goes to a PLC input, so it stops the belt through the program. A real Emergency stop should also cut the drive's power or enable signal by hardware. Copy the old machine as it is, but know this limit, and ask an electrician before anyone relies on it for safety.

5. From a problem statement to a program: the method

This is the order an experienced PLC programmer works in. The old program follows it, and you can use it for any new machine. Do steps 1 to 7 on paper **before** you open TIA Portal. Each step below shows the method, then the conveyor as the worked example.

Step 1. Write the problem statement in plain words

Write what the machine must do, as the operator sees it. No PLC words yet. For your machine (from your explanation, 6 Oct 2026):

A conveyor carries stacks of boxes past a QR camera. The operator chooses AUTO or MANUAL with a selector switch.
AUTO: the green START button starts the cycle and the STOP button ends it. While the cycle runs, the belt moves forward, unless the PC says stop. The PC counts the QR codes the camera reads; if the count is wrong it stops the belt. The operator restarts from the PC.
MANUAL: the belt moves forward or backward only while the operator holds a button (jog).
Faults: the Emergency button or a drive fault stops everything, shows an alarm and a red lamp, and needs a reset after the cause is gone.
Lamps: green = auto cycle running, amber = manual mode, red = fault. The panel shows mode, status, alarms and the state of every input and output.

Step 2. List every device and decide its inputs and outputs (the I/O list)

Go through the machine device by device and ask: **what does the PLC need to know from it** (input) and **what must the PLC tell it** (output)? One wire = one input or one output.

Device	Signal	Direction	Why the PLC needs it	Address
Selector switch	AUTO / MANUAL	Input	Choose the mode	I0.1
Green button	START	Input	Start the auto cycle; jog forward in manual	I0.2
Stop button (NC)	STOP	Input	End the auto cycle	I0.3
Emergency (NC)	Healthy	Input	Stop everything, raise alarm	I0.4
VFD relay	Fault	Input	Stop everything, raise alarm	I0.5
VFD	Run forward	Output	Move the belt forward	Q0.0
VFD	Run reverse	Output	Move the belt backward (manual)	Q0.1
VFD	Reset fault	Output	Clear a drive fault from the HMI	Q0.2
Tower light	Green / red / amber	3 outputs	Show the state	Q0.3 / Q0.4 / Q0.5

Not in the list: the IR sensor (it goes to the camera, not the PLC) and the motor speed (set in the VFD). I0.0 still has a tag because the program was first written with the sensor on the PLC (part 14).

Step 3. List what comes over the network

Some signals have no wire: they come from the HMI or the PC over Ethernet. List them the same way.

From	To	Signal	Kept in
HMI	PLC	Jog forward button, jog reverse button, fault reset button	DB_HMI
PLC	HMI	Mode, auto status, alarms, state of all inputs and outputs	M2.2, DB_AUTO, DB_ALARMS, IB0 / QB0
PC	PLC	Run allowed (0) or stop (1)	DB_CAMERA.DATA1 (register 40001)

Step 4. Write every rule as "X is ON when ..."

Take each output and each important state, and write one sentence that says exactly when it is ON. This is the most important step: **each sentence later becomes one network**.

#	Result	is ON when ...	Network
R1	Control ON	the Emergency button is not pressed	FC_GENERAL 1
R2	Manual mode / Auto mode	selector OFF / selector ON	FC_GENERAL 2
R3	Auto cycle ACTIVE	control ON **and** AUTO **and** no fault **and** STOP not pressed **and** (START pressed **or** already ACTIVE)	FC_AUTO 1
R4	Auto wants the belt to run	ACTIVE **and** the PC value DATA1 = 0	FC_AUTO 3 (row 1)
R5	Manual forward	control ON **and** MANUAL **and** no fault **and** (HMI FWD jog held **or** START held)	FC_MANUAL 1
R6	Manual reverse	control ON **and** MANUAL **and** no fault **and** HMI REV jog held	FC_MANUAL 1
R7	Emergency alarm	**set** when Emergency is pressed; stays until reset	FC_ALARMS 1
R8	Drive alarm	**set** when the VFD relay reports a fault; stays until reset	FC_ALARMS 2
R9	All alarms cleared	the HMI FAULT RESET button is held	FC_ALARMS 3
R10	FAULT BIT (any fault)	any alarm bit is 1	FC_ALARMS 4
R11	Q0.0 forward	auto wants to run **or** manual forward	FC_OUTPUTS 1
R12	Q0.1 / Q0.2	manual reverse / reset button held	FC_OUTPUTS 2, 3
R13	Green / red / amber	ACTIVE / FAULT BIT / manual mode	FC_OUTPUTS 4, 5, 6

Step 5. Find what the PLC must remember

Look at each rule and ask: **does the result depend only on what is happening now, or also on what happened before?**

- **The auto cycle (R3)** must be remembered: START is a push button, it is 1 only while the finger is on it. After release, the PLC must still know "the cycle was started". Solution: a **latch** (seal-in): ACTIVE keeps itself on through its own contact.
- **The alarms (R7, R8)** must be remembered: the operator must see that an Emergency happened even after the button is released. Solution: **set coils**, cleared only by the reset (R9).
- **The PC's decision (R4)** is remembered by the PLC in DATA1 until the PC writes again.
- Everything else (mode, jog, lamps) depends only on the present moment: plain coils.

Step 6. Group the rules into jobs = program blocks

Job	Rules	Block
General conditions that every mode needs	R1, R2, reset button	FC_GENERAL (FC1)
What happens in manual mode	R5, R6	FC_MANUAL (FC2)
Detect, remember and clear faults	R7 - R10	FC_ALARMS (FC3)
Drive the real outputs	R11 - R13	FC_OUTPUTS (FC4)
What happens in auto mode	R3, R4, status text	FC_AUTO (FC5)
Talk to the PC	Modbus server	FC_TCP_IP (FC6)

Small blocks with one job each are easier to read, test and change. You could put everything in OB1, and it would work, but nobody could find anything.

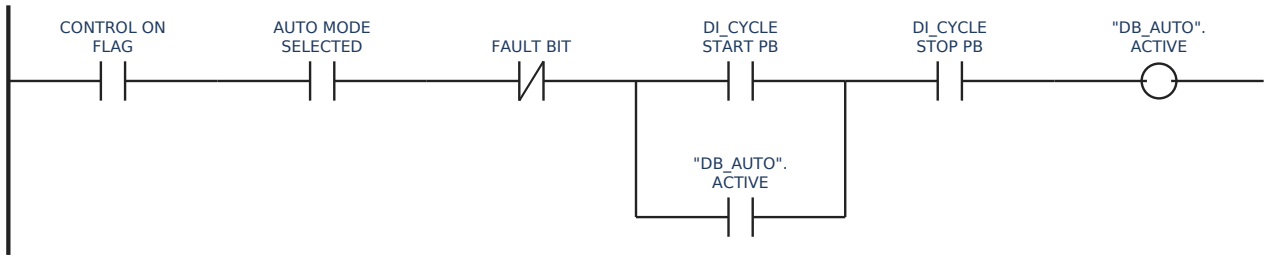
Step 7. Choose names and where each value lives

- **Inputs and outputs:** a tag for each, with a prefix that tells what it is: **DI_** = digital input, **DO_** = digital output, **TL** = tower light, **PB** = push button.
- **Decisions that one block makes and other blocks use:** M flags (part 6).
- **Values that belong to one subject** (HMI buttons, alarms, auto cycle, PC data, connection): one DB per subject (part 7).
- Write the list on paper first. Changing a name later is easy in TIA; changing an address after the HMI uses it is not.

Step 8. Turn each rule into one network

Read the sentence and draw it: **and** = contacts in a row, **or** = contacts in a branch, **not** = normally closed contact, **remember** = latch or set coil. Example, rule R3:

R3 drawn: ACTIVE



control ON **and** AUTO **and not** fault **and** (START **or** ACTIVE) **and** STOP-input (1 while not pressed) = ACTIVE. Word by word, the sentence becomes the picture.

Step 9. Design the HMI from the operator's tasks

List what the operator must **do** (jog, reset) and **see** (mode, status, alarms, I/O). Each task becomes a button or a field; related things go on one screen; things needed everywhere go on the template (part 12).

Step 10. Write the test list from the rules, then test

Every rule from step 4 becomes at least one test: "do this, you must see that". The test list in part 11 of the how-to guide is made this way. Test with the motor disconnected or the belt empty first, and use the monitoring view (glasses icon) to watch each contact.

6. Why these tags, and why this many

The old project has **three tag tables** with **31 own tags**. The number is not chosen; it follows from the steps above:

- **DI table, 14 tags:** 6 used inputs (I0.0 - I0.5: one per wire in the I/O list, plus I0.0 for the sensor that is now on the camera) and 8 spares (DI_SPARE I0.6 ... DI_SPARE I1.5). The CPU has 14 inputs, so every input has a name.
- **DO table, 10 tags:** 6 used outputs (Q0.0 - Q0.5) and 4 spares (DO_SPARE Q0.6 ... DO_SPARE Q1.1): every one of the 10 outputs has a name.
- Naming every spare is a good habit: when a new wire is added later, you only rename the tag (part 12 of the how-to guide does exactly that with I0.7).
- **7 flags** (M2.0 - M2.6): one for each decision that is made in one block and used in another. A flag is how blocks talk to each other.

Flag	Written by (one place)	Read by	Why it is a flag
M2.0 CONTROL ON FLAG	FC_GENERAL network 1	FC_MANUAL, FC_AUTO	"Emergency is healthy" is needed in both modes: decide once, use twice.
M2.1 MANUAL MODE SELECTED	FC_GENERAL network 2	FC_MANUAL, FC_OUTPUTS (amber)	Gives the mode a clear name instead of "selector input is 0".
M2.2 AUTO MODE SELECTED	FC_GENERAL network 2	FC_AUTO, HMI mode text	Same, for AUTO. The HMI reads it too.
M2.3 FAULT RESET FLAG	FC_GENERAL network 3	FC_ALARMS (clear), FC_OUTPUTS (Q0.2)	One reset signal for the whole program. A reset button on the door could be added in one place later.
M2.4 FAULT BIT	FC_ALARMS network 4	FC_MANUAL, FC_AUTO, FC_OUTPUTS (red)	"Any alarm" summarised in one bit.
M2.5 MANUAL CONVEYOR FWD ON FLAG	FC_MANUAL	FC_OUTPUTS network 1	Manual's wish; FC_OUTPUTS combines it with auto's wish.
M2.6 MANUAL CONVEYOR REV ON FLAG	FC_MANUAL	FC_OUTPUTS network 2	Same, for reverse.

Why start at M2.0, not M0.0? MB0 is the clock memory byte (switched on in the old project, M0.0 - M0.7) and MB1 the system memory byte (M1.0 - M1.3). So the first free byte is MB2. Any free byte would work; what matters is that nothing else uses it.

System tags (created by the CPU because the system memory byte is switched on at MB1): FirstScan M1.0, DiagStatusUpdate M1.1, **AlwaysTRUE M1.2** and **AlwaysFALSE M1.3**. The program uses the last two: AlwaysTRUE as a contact that always passes (to give a network a starting contact), AlwaysFALSE as a fixed 0 for the Modbus DISCONNECT pin.

7. Why these data blocks

Flags are fine for single bits that blocks pass around. Values that belong together, or that another device (HMI, PC) must reach, are kept in data blocks: **one DB per subject**, so you always know where to look.

DB	Subject	Why it exists	Why this size
DB_HMI (DB1)	Values the HMI writes	Keeps everything the operator can change from the panel in one place, apart from the PLC's own decisions.	3 buttons. STOP TIME and CAMERA DONE belong to the old sensor cycle.
DB_ALARMS (DB2)	Alarms	The HMI alarm system needs the alarm bits in one **word**, and the reset clears that word in one MOVE.	2 real alarms + 14 spares = 16 bits = exactly one word (DBW0). Spares are room for future alarms. Then FAULT POP / WARNING POP.
DB_AUTO (DB3)	The auto cycle's memory	Everything FC_AUTO must remember (ACTIVE, CONVEYOR ON, status) in one place.	3 values used now; the rest (OFF1, OFF2, timers ...) belong to the old sensor cycle.
MB_SERVER_DB (DB4)	Private memory of the Modbus server	MB_SERVER is an FB; an FB always needs an instance DB. TIA makes it when you drop MB_SERVER into the network.	Decided by Siemens. Do not edit.
DB_CAMERA (DB5)	Data shared with the PC	The Modbus server needs one block of Int values with fixed offsets: these are the PC's "registers".	19 Ints, but the server shows only the first 10 to the PC (DATA1 - DATA10 = 40001 - 40010), and only DATA1 is used. The extra rows are spare room; there is no technical reason for 19.

DB	Subject	Why it exists	Why this size
DB_TCP_IP (DB12)	The Modbus connection	Connection settings (TCON_IP_v4) and fault-finding counters. The only **optimized** DB: it is used only by name.	Settings + 3 counters. The number 12 is the author's choice; any free number works.

7.1 Why the DB numbers matter

Inside the PLC program you use names, so the numbers would not matter. But two things use **numbers and offsets**: the alarm word **%DB2.DBW0** (typed as an address in FC_ALARMS and in the HMI tag ALARM WORD), and the Modbus area **P#DB5.DBX0.0 INT 10**. If your DB_ALARMS became DB3, those addresses would point at the wrong data. That is why the how-to guide makes you type the numbers by hand.

7.2 Why the offsets jump (0.2, then 2.0)

In a non-optimized DB, Bools are packed bit by bit, but any value bigger than a Bool starts at the next **even** byte. In DB_HMI the three buttons use byte 0 (0.0, 0.1, 0.2), so STOP TIME (DInt, 4 bytes) starts at 2.0 and fills bytes 2 - 5, and CAMERA DONE lands on 6.0. In DB_AUTO a timer (IEC_TIMER) uses 16 bytes: 4.0 to 19.x, so the next one starts at 20.0. You do not type offsets; TIA calculates them when you compile. Comparing them with the guide is a quick check that you typed every row and type correctly.

7.3 Why "Optimized block access" is switched off on four DBs

With optimized access a DB has no fixed layout, so addresses like DB2.DBW0 do not exist and the Modbus server cannot be pointed at DB5. DB_ALARMS and DB_CAMERA need fixed addresses, and the author also made DB_HMI and DB_AUTO non-optimized (the HMI and the old sensor cycle used them). **DB_TCP_IP stays optimized**: nothing reaches it by address, only by name. Optimized is faster and is the TIA default.

8. Why six blocks, and why this order in OB1

OB1 calls the blocks in this order (read from the old file): **FC_AUTO, FC_ALARMS, FC_GENERAL, FC_MANUAL, FC_OUTPUTS, FC_TCP_IP**.

- **FC_OUTPUTS near the end** is deliberate and important: by then every block has made its decision in this scan, so the outputs show the newest result.
- **FC_TCP_IP last:** it does not feed any other block in the same scan, so its place does not matter. MB_SERVER must be called in **every** scan to keep the connection alive; calling it from OB1 does that.
- **FC_AUTO first, FC_GENERAL third** has no clear reason. It means FC_AUTO uses the mode, control and fault flags from the previous scan. That is a few milliseconds late, which nobody can notice on a conveyor, so it works. If you design your own program, a clearer order is: inputs and modes first (FC_GENERAL), then alarms, then the modes (FC_AUTO, FC_MANUAL), then outputs.

For the new machine, **copy the old order exactly**. The goal is an identical machine, and this order is proven on the real conveyor. Try the clearer order only in a practice copy.

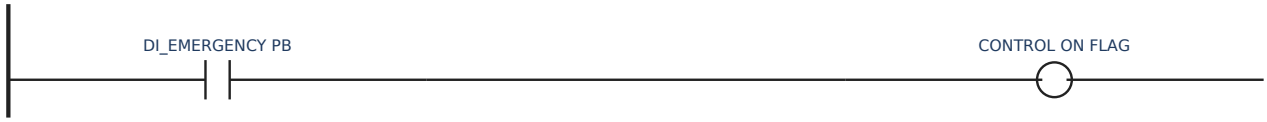
Why FCs and not FBs? An FB is useful when the same logic is used several times, each copy with its own memory (for example three identical conveyors). Here each job exists once and all memory is in the global DBs, so simple FCs are enough.

9. Every network explained

For each network: the question it answers, and **why each contact is there** (what would go wrong without it).

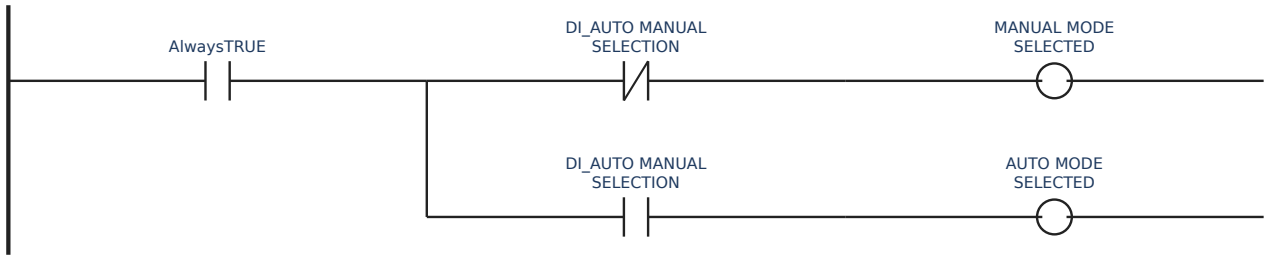
FC_GENERAL (FC1): general conditions

Network 1: "Is the machine allowed to work?"



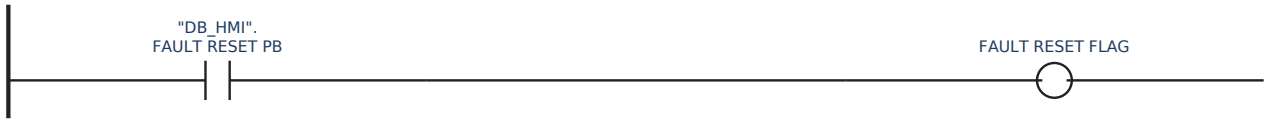
The input is 1 while Emergency is **not** pressed (NC wiring), so a normally open contact gives "healthy". Every movement later asks this flag, so one Emergency press stops both modes.

Network 2: "Which mode?"



One selector input, two opposite flags: NC contact = manual, NO contact = auto. Exactly one of them is always 1, so the program can never be in both modes. AlwaysTRUE gives the two branches a common starting contact; it is a drawing habit and changes nothing.

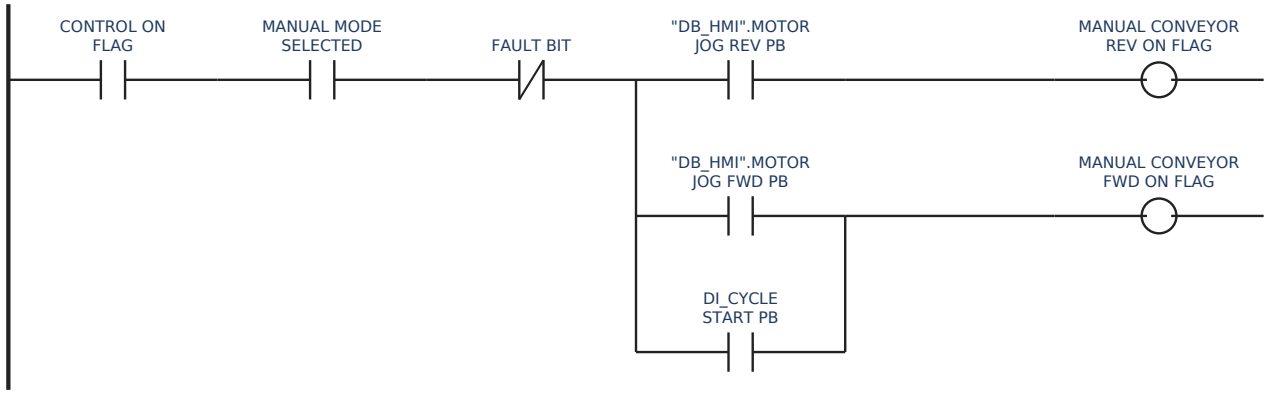
Network 3: "Is reset requested?"



Copies the HMI button into a PLC flag, so the rest of the program uses one name for "reset", whatever the source.

FC_MANUAL (FC2): jogging

Network 1: "Should the belt jog, and which way?"

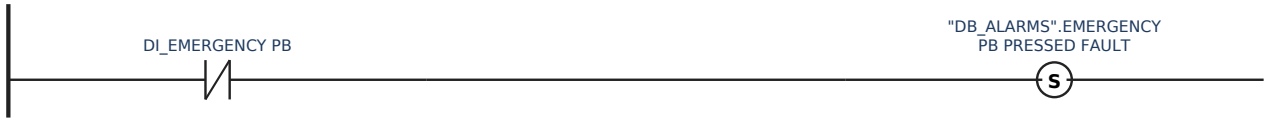


CONTROL ON: no jogging during Emergency. MANUAL: jogging must not work in AUTO. FAULT BIT (NC): no jogging with an alarm. Then the button decides the direction. There is **no latch**: plain coils, so the belt moves only while a button is held (that is what "jog" means).

Something the old program does not do: there is no interlock between forward and reverse. Holding the green START and the HMI REV JOG together turns on both Q0.0 and Q0.1. Many drives stop when they get both commands, but do not rely on it. Exercise 3 (part 13) shows how to add the interlock in a practice copy. The **red STOP button has no job in MANUAL** in the old program; the red-button reverse jog you asked for is part 12 of the how-to guide.

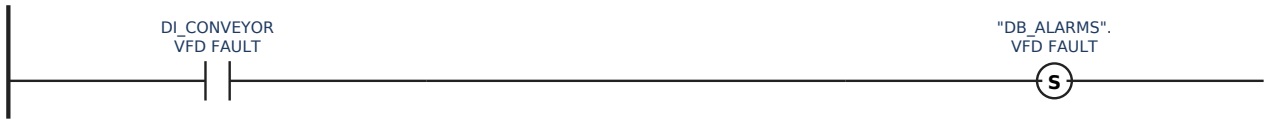
FC_ALARMS (FC3): faults

Network 1: "Was Emergency pressed?"



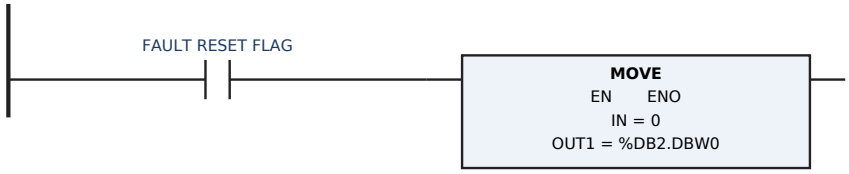
Here the NC symbol is used on purpose: the input is 0 when pressed, and the NC symbol passes on 0. Set coil: the alarm stays after the button is released, so the operator sees what happened and must reset it.

Network 2: "Did the drive report a fault?"



The VFD relay closes on a fault (parameter 03.00 on the old drive = 8), so the input is 1 = fault.

Network 3: "Clear all alarms"



Writing 0 to the whole word clears all 16 alarm bits in one go, including any spare alarm you add later. If the cause is still there (Emergency still pressed), network 1 sets the alarm again as soon as reset is released: you cannot hide a fault that is still present.

Network 4: "Is there any fault?"



If any of the 16 bits is 1, the word is not 0. One compare checks all alarms, present and future. FAULT BIT goes to the program and the red lamp; FAULT POP goes to the HMI to show the red alarm line.

FC_OUTPUTS (FC4): the only place that writes outputs

Network 1: forward



Both modes may want forward. If FC_MANUAL and FC_AUTO each wrote Q0.0 with their own coil, the last one in the scan would win and the other would be ignored (part 2). Here the two wishes are joined with OR and Q0.0 is written once.

Network	Output	ON when	Why
2	Q0.1 reverse	manual reverse flag	Only manual can reverse

Network	Output	ON when	Why
3	Q0.2 reset to VFD	reset flag	Pulses the drive's reset input while the HMI button is held (only works if the wire exists on the panel)
4	Q0.3 green	"DB_AUTO".ACTIVE	Green = auto cycle started, even while the PC holds the belt
5	Q0.4 red	FAULT BIT	Red = any alarm
6	Q0.5 amber	manual mode	Amber = manual

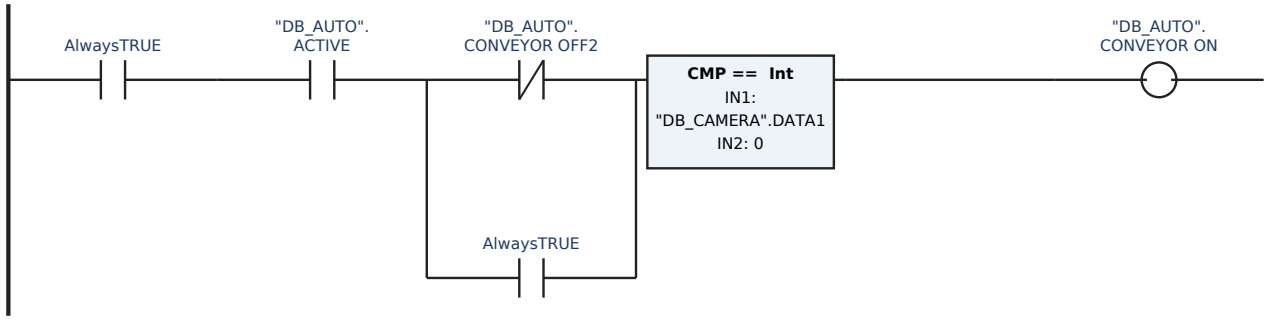
FC_AUTO (FC5): the auto cycle

Network 1 is rule R3, drawn in part 5 step 8. Why each contact:

Contact	Without it ...
CONTROL ON FLAG	START would work while Emergency is pressed.
AUTO MODE SELECTED	START would start the cycle in manual mode; switching to MANUAL would not end the cycle.
FAULT BIT (NC)	The cycle would keep running, or could be started, with an active alarm.
START with ACTIVE in parallel	This is the latch. Without the ACTIVE branch, the cycle would end when you let go of START.
DI_CYCLE STOP PB (NO symbol, NC button)	STOP would do nothing. Because it is in series after the latch, pressing STOP breaks the latch.

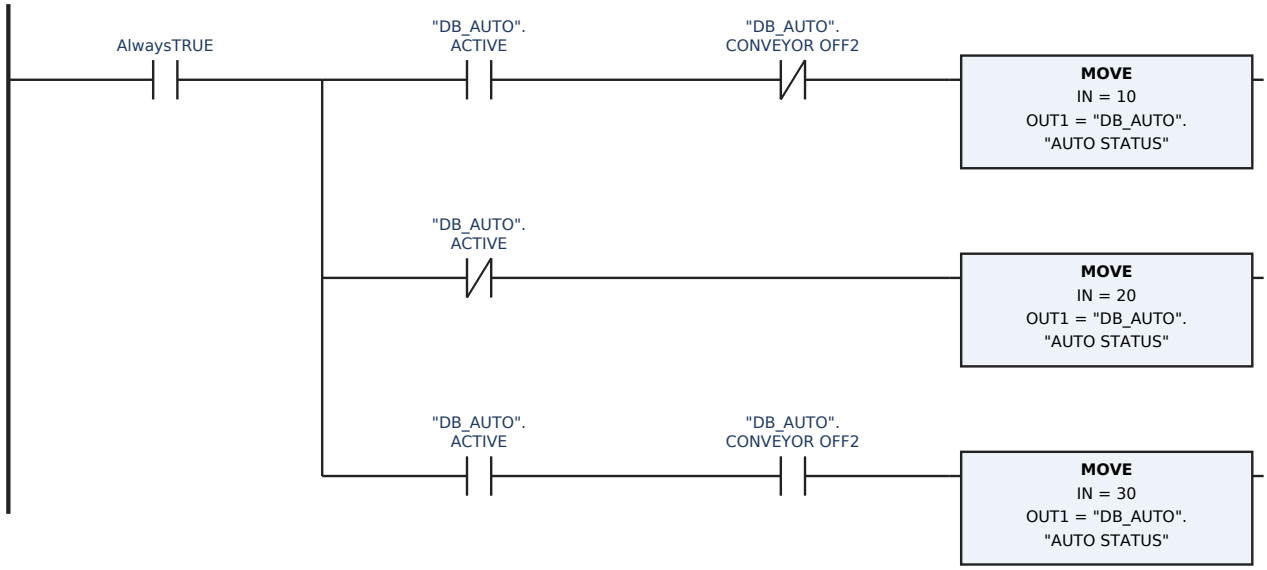
The old FC_AUTO has six networks. Only networks 1, 3 (first row) and 6 do work on your machine; the rest is the old sensor cycle, switched off (how-to guide, Appendix A).

Network 3, first row: "Should the belt run now?"



Two real conditions: the operator started the cycle (ACTIVE), and the PC allows running (DATA1 = 0). The NOT OFF2 contact belonged to the old cycle; the AlwaysTRUE in parallel makes it pass always. Note the split of responsibility: ACTIVE (green lamp) stays ON when the PC stops the belt, so the operator sees "cycle started, belt held by the PC".

Network 6: "What should the HMI say?"



The PLC sends a **number**, the HMI turns it into text with the text list AUTO_STATUS (10 = AUTO RUNNING, 20 = STOPPED, 30 = WAITING FOR CAMERA ACK, which never happens now). Numbers are small and fast to send, and texts can be changed on the HMI without touching the PLC.

Networks 2, 4, 5 and the lower rows of network 3 are the switched-off sensor cycle: a stop-delay timer, a sensor timer, an edge on I0.0, and the OFF1 / OFF2 memories. A **normally closed AlwaysTRUE** contact at the start of that part means it can never run.

FC_TCP_IP (FC6): the Modbus server

- **Network 1** fills the connection settings with MOVE boxes in every scan. Writing the same values again costs nothing and guarantees they are right.
- **Network 2** calls MB_SERVER. It listens on port 502 and maps the PC's registers onto DB5 (part 11).
- **Network 3** counts good messages (NDR = new data received, DR = data read) and errors, and keeps the last error code. The machine works without them; they exist so you can see in a watch table whether the PC is really talking to the PLC.

10. Follow a signal through the program

These four stories show how the blocks work together. Try each one on the real machine with the monitoring view open.

Story 1: AUTO, operator presses START

1. I0.2 becomes 1. In the next scan FC_AUTO network 1 sees START, together with control ON, AUTO and no fault: **ACTIVE = 1**. The ACTIVE branch now holds it.
2. FC_AUTO network 3: ACTIVE = 1 and DATA1 = 0 (start value): **CONVEYOR ON = 1**. Network 6 writes 10 to AUTO STATUS.
3. FC_OUTPUTS: CONVEYOR ON gives **Q0.0 = 1** (VFD runs forward). ACTIVE gives **Q0.3 = 1** (green lamp).
4. The HMI reads AUTO STATUS = 10 and shows AUTO RUNNING. The operator releases START: nothing changes, the latch holds.

Story 2: the PC finds a wrong count

1. The PC program sends Modbus "write single register", address 0, value 1. MB_SERVER writes 1 into DB5.DBW0 = DATA1.

2. FC_AUTO network 3: DATA1 is no longer 0, so CONVEYOR ON = 0. FC_OUTPUTS: **Q0.0 = 0**, the belt stops.
3. ACTIVE is still 1: green lamp stays on, HMI still says AUTO RUNNING.
4. The operator fixes the stack and presses Start (or Rescan) on the PC. The PC writes 0, and the belt runs again.

Story 3: Emergency pressed, then reset

1. I0.4 becomes 0. FC_ALARMS network 1 **sets** the Emergency alarm. Network 4: the word is not 0, so **FAULT BIT = 1**.
2. FC_GENERAL: CONTROL ON = 0. FC_AUTO network 1 now fails (no control, fault): **ACTIVE = 0**, the latch is broken. Q0.0 = 0, the belt stops.
3. FC_OUTPUTS: red lamp ON, green OFF. The HMI shows the red line and the alarm text "EMERGENCY PB PRESSED FAULT (I0.4)".
4. Operator releases Emergency: the alarm **stays** (set coil). Operator presses FAULT RESET on the HMI: network 3 writes 0 to DBW0, FAULT BIT = 0, red lamp OFF.
5. The cycle does **not** restart by itself: ACTIVE was broken. The operator must press START again. This is intentional: a machine must not start on its own after a fault.

Story 4: MANUAL, operator holds CONVEYOR REV JOG on the HMI

1. The HMI button event "Press" runs SetBitWhileKeyPressed: DB1.DBX0.2 (MOTOR JOG REV PB) = 1, and the button turns green.
2. FC_MANUAL: control ON, MANUAL, no fault, REV jog: **REV flag = 1**. FC_OUTPUTS: **Q0.1 = 1**, the belt runs backward.
3. Operator lifts the finger: SetBitWhileKeyPressed clears the bit again, Q0.1 = 0, the belt stops.

11. Modbus TCP, the PC-to-PLC conversation

Word	Meaning	In this project
Modbus TCP	A simple, very common language for machines to read and write numbers over Ethernet.	PC to PLC
Server / client	The server waits and answers; the client connects and asks. (Older word: slave / master.)	PLC = server, PC = client
Port 502	The standard network "door" number for Modbus TCP.	LocalPort = 502
Holding register	A 16-bit number the client may read and write. Written in the "4xxxx" numbering: the first one is 40001, which is address 0 in the message.	40001 = DATA1
Function 06	"Write single register".	What the PC sends (file server.py)
MB_HOLD_REG	The PLC memory area the server shows as registers.	P#DB5.DBX0.0 INT 10: 10 Ints from DB5, byte 0

The mapping: register 40001 = first Int of the area = DB5.DBW0 = "DB_CAMERA".DATA1. Register 40002 = DATA2, and so on up to 40010.

Why the PLC is the server: the PLC is always on and always at 192.168.0.40; the PC program starts later and connects when it needs to. That is why the connection settings say remote address 0.0.0.0 (accept any client), ActiveEstablished = 0 (wait, do not call out), and the PLC never needs to know the PC's address.

When the PC writes: 0 or 1 when the operator presses Start / Stop in the PC program, and 1 automatically when the number of QR codes read does not match the expected number (from the PC program's server.py).

12. The HMI, explained

Checked against the HMI exported by TIA Portal on 12 October 2026.

- **HMI tags** are the panel's only window into the PLC. There are 15, in three tables: **Default** (buttons, mode, status, alarm pop-ups, plus two tags of the old sensor cycle and the internal Tag_ScreenNumber), **IO** (IB0, IB1, QB0, QB1) and **ALARMS** (ALARM WORD). Each screen element needs only one value.
- **Template MAIN:** the title bar, mode line and the five buttons are the same on every screen, so they are drawn once on the template. Change the template and every screen follows.
- **Buttons use SetBitWhileKeyPressed on Press.** The bit is 1 only while the finger is on the button, like a real push button. That is exactly what jog and reset need. The jog buttons also turn **green** while pressed (an appearance animation on the same tag).
- **I/O status screens read whole bytes** (IB0, IB1, QB0, QB1), and each lamp looks at one bit of its byte: **red = OFF, green = ON**. Four tags show 24 signals.
- **Text lists** turn numbers into words: AUTO STATUS 10 / 20 / 30, MODE 0 / 1. The AUTO status field also changes colour: **green** at 10 (AUTO RUNNING), **red** at 20 (STOPPED).
- **Discrete alarms** watch one bit each in ALARM WORD. Bits 8 and 9, not 0 and 1, because in a Siemens word the first byte (DBB0) is the **high** half: DBX0.0 is bit 8 of DBW0 and DBX0.1 is bit 9. (The export cannot include alarms; these come from older files of the project.)
- **Acquisition mode "Cyclic continuous"** for ALARM WORD: the panel keeps reading it even when no screen shows it, so alarms appear on any screen. All other tags use "Cyclic in operation": read only while a screen shows them.
- **One mistake in the old HMI:** the BACK button on OUTPUT STATUS opens OUTPUT STATUS itself, so it does nothing. The how-to guide (10.5) shows the fix.

13. Practise: three small exercises

Do these only in a COPY of your project (Project > Save as ... Conveyor_Practice), never in the project you load into the machine. The machine program must stay identical to the old one. Use S7-PLCSIM or a PLC on the bench with no motor connected.

Exercise 1. Show the Modbus message counter on the HMI

1. HMI tags: add **MODBUS DONE**, PLC tag "DB_TCP_IP"."NO OF DONE" (data type UDInt).
2. AUTO screen: add a text field "PC MESSAGES" and an I/O field next to it, mode **Output**, tag MODBUS DONE.
3. Compile, load, and watch the number count up each time the PC sends Start or Stop.

You learn: how a PLC value reaches the panel (DB value, HMI tag, field).

Exercise 2. Make the amber lamp blink in MANUAL

1. The clock memory byte is already switched on at address **0** in this project (PLC properties > **System and clock memory**). It gives tags such as **Clock_1Hz = M0.5** (1 on, 1 off, once per second).
2. FC_OUTPUTS network 6: put a normally open contact **Clock_1Hz** in series after MANUAL MODE SELECTED.
3. Compile and test: in MANUAL the amber lamp now blinks.

You learn: clock memory, which the old project switched on but never used (part 6).

Exercise 3. Add a forward / reverse interlock

1. FC_OUTPUTS network 1: add a **normally closed** contact **DO_CONVEYOR REV ON** just before the coil DO_CONVEYOR FWD ON.
2. Network 2: add a normally closed contact **DO_CONVEYOR FWD ON** before the coil DO_CONVEYOR REV ON.
3. Test: in MANUAL hold START and REV JOG together. Only one output can be ON now.

You learn: interlocks, and the scan order: network 1 runs first, so if both are requested in the same moment, forward wins.

Check yourself

Question	Answer
The STOP button is NC. Why does the program use a normally OPEN contact for it?	The input is 1 while STOP is not pressed. The NO symbol passes on 1, so the rung works until STOP is pressed.
Why can Q0.0 not be written in both FC_MANUAL and FC_AUTO?	The last coil in the scan wins, so one mode would be ignored. Both wishes are joined in FC_OUTPUTS.
After an Emergency and reset, why does the belt not restart by itself?	ACTIVE was broken by the fault; START must be pressed again. This is wanted for safety.
Which DB does register 40001 write to, and at what address?	DB_CAMERA (DB5), DATA1 at DBW0.
Why must DB_ALARMS have "Optimized block access" switched off?	FC_ALARMS and the HMI use the absolute address DB2.DBW0, which only exists in a non-optimized DB.
Why is there an instance DB4 that you never created?	MB_SERVER is an FB; TIA creates its instance DB when you place it.
The green lamp is ON but the belt stands still in AUTO. Why?	ACTIVE = 1 but DATA1 is not 0: the PC has stopped the belt.

14. Honest notes: fact, inference and limits

Statement	Source
Tag names, addresses, DB layouts, networks, the OB1 order	Exported from the old project by TIA Portal itself on 12 Oct 2026 (fact)
How the machine is operated (AUTO / MANUAL, PC stops the belt)	Your explanation of 6 Oct 2026, matches the program and the PC code
When the PC writes 0 / 1 to 40001	Read from the PC program (Frontend/backend/server.py)
VFD relay = fault (03.00 = 8)	Read from the old drive on 7 Oct 2026; meaning of 8 to be confirmed in the Delta manual
Reasons for M2 start, DB12, 19 DATA rows, OB1 order	Inference: usual practice, or no technical reason, as stated in each part
I0.0 tag and the extra rows in DB_HMI / DB_AUTO	Left over from the first version where the sensor was on the PLC (how-to guide, Appendix A)

Open points that affect the program (also in the project README):

- You said the red button reverses the belt in MANUAL. The saved file has reverse only on the HMI button. Test it on the old machine; if true, the old PLC holds a newer program: upload it from the PLC instead of guessing.
- Q0.2 (reset to the VFD): check on the old panel whether a wire goes from Q0.2 to a VFD input.